# JWT Authentication in Laravel

3-Day Practical Task for Junior Developers

| ■■■ INTERMEDIATE | Duration: 3 Days | Experience: 1+ Month |
|---|---|---|

## ■ You Should Already Know:

• Laravel basics (creating controllers, models, routes)
• Database concepts (migrations, relationships)
• REST APIs and HTTP methods
• How to use Postman for testing
• Basic password hashing

## ■ What You'll Learn:

→ How JWT tokens work and why they're used
→ Implement user registration and login with JWT
→ Manage token expiration and refresh
→ Protect API routes with authentication
→ Handle authentication errors properly
→ Test APIs using Postman
→ Store user roles and basic permissions

## ■ DAY 1: Setup & Authentication API (6 hours)

### *Task 1.1: Project Setup (1 hour)*

**What to Do:**
→ Create new Laravel project: `laravel new jwt-auth`
→ Install JWT package: `composer require tymon/jwt-auth`
→ Publish config: `php artisan vendor:publish --provider='Tymon\\JWTAuth\\Providers\\LaravelServiceProvider'`
→ Generate secret: `php artisan jwt:secret`
→ Update .env with database credentials

### *Task 1.2: Setup User Model & Database (1.5 hours)*

**What to Do:**
1. Update User Model:

```
id; } public function getJWTCustomClaims() { return []; } }
```

2. Run migrations:

```
php artisan migrate
```

3. Update config/auth.php:

```
'guards' => [ 'api' => [ 'driver' => 'jwt', 'provider' => 'users', ], ],
```

### *Task 1.3: Create Authentication Routes (1.5 hours)*

**What to Do:** Create routes/api.php with auth endpoints

```
Route::prefix('auth')->group(function () { Route::post('/register', [AuthController::class, 'register']); Route::post('/login', [AuthController::class, 'login']); Route::middleware('auth:api')->group(function () { Route::get('/me', [AuthController::class, 'me']); Route::post('/logout', [AuthController::class, 'logout']); Route::post('/refresh', [AuthController::class, 'refresh']); }); });
```

### *Task 1.4: Build Authentication Controller (2 hours)*

**Endpoints Needed:**
• POST /api/auth/register - Create new user
• POST /api/auth/login - Login and get token
• GET /api/auth/me - Get user profile
• POST /api/auth/logout - Logout
• POST /api/auth/refresh - Refresh token

**Register Method Example:**

```
public function register(Request $request) { $validated = $request->validate([ 'name' => 'required|string', 'email' => 'required|email|unique:users', 'password' => 'required|min:6|confirmed', ]); $validated['password'] = Hash::make($validated['password']); $user = User::create($validated); $token = auth('api')->login($user); return response()->json([ 'status' => 'success', 'message' => 'User registered', 'token' => $token, 'user' => $user, ], 201); }
```

**Login Method Example:**

```
public function login(Request $request) { $credentials = $request->validate([ 'email' => 'required|email', 'password' => 'required', ]); if (!auth('api')->attempt($credentials)) { return response()->json([ 'status' => 'error', 'message' => 'Invalid credentials', ], 401); } $token = auth('api')->attempt($credentials); return response()->json([ 'status' => 'success', 'message' => 'Login successful', 'token' => $token, 'user' => auth('api')->user(), ]); }
```

## *Day 1 Deliverables:*

✓ JWT package installed and configured
✓ User model with JWT implementation
✓ All 5 API endpoints working
✓ Database setup with users table

## ■ DAY 2: Error Handling & Validation (6 hours)

### *Task 2.1: Complete Other Auth Methods (2 hours)*

**Implement these methods in AuthController:**

**1. Get User Profile (me):**

```
public function me() { return response()->json([ 'status' => 'success', 'user' =>
auth('api')->user(), ]); }
```

**2. Logout:**

```
public function logout() { auth('api')->logout(); return response()->json([ 'status' =>
'success', 'message' => 'Logged out successfully', ]); }
```

**3. Refresh Token:**

```
public function refresh() { try { $token = auth('api')->refresh(); return response()->json([
'status' => 'success', 'message' => 'Token refreshed', 'token' => $token, ]); } catch
(TokenExpiredException $e) { return response()->json([ 'status' => 'error', 'message' =>
'Token expired', ], 401); } }
```

### *Task 2.2: Add Error Handling (2 hours)*

**Handle these errors in AuthController:**

→ TokenExpiredException - Token has expired
→ TokenInvalidException - Token is invalid or tampered
→ JWTException - General JWT errors
→ ValidationException - Input validation fails

**Example Error Handler:**

```
try { $user = auth('api')->userOrFail(); } catch (JWTException $e) { return response()->json([
'status' => 'error', 'message' => 'Token not valid', ], 401); } catch (TokenExpiredException
$e) { return response()->json([ 'status' => 'error', 'message' => 'Token expired', ], 401); }
```

### *Task 2.3: Create Resource Class (1 hour)*

**What to Do:** Create UserResource to format user responses

```
// app/Http/Resources/UserResource.php $this->id, 'name' => $this->name, 'email' =>
$this->email, 'created_at' => $this->created_at, ]; } }
```

### *Day 2 Deliverables:*

✓ All 5 auth methods fully implemented
✓ Proper error handling for all cases
✓ UserResource for consistent responses
✓ Tested endpoints in Postman

## ■ DAY 3: Testing & Basic Features (6 hours)

### *Task 3.1: Basic Authorization & Roles (2 hours)*

**What to Do:** Add simple role-based access control

1. Add role column to users table (migration):

```
Schema::table('users', function (Blueprint $table) { $table->string('role')->default('user');
// admin or user });
```

2. Create simple middleware:

```
// app/Http/Middleware/IsAdmin.php public function handle($request, Closure $next) { if
(auth('api')->user()->role !== 'admin') { return response()->json([ 'status' => 'error',
'message' => 'Only admins allowed', ], 403); } return $next($request); }
```

3. Use in routes:

```
Route::post('/admin/users', [AdminController::class, 'listUsers']) ->middleware(['auth:api',
'isAdmin']);
```

### *Task 3.2: Password Reset Feature (2 hours)*

**What to Do:** Implement password reset endpoints

Create 2 endpoints:

• POST /api/auth/forgot-password - Send reset link via email

• POST /api/auth/reset-password - Reset with token

**Forgot Password Method:**

```
public function forgotPassword(Request $request) { $request->validate(['email' =>
'required|email|exists:users']); $token = Str::random(60);
DB::table('password_resets')->insert([ 'email' => $request->email, 'token' =>
Hash::make($token), 'created_at' => now(), ]); // TODO: Send email with reset link return
response()->json([ 'status' => 'success', 'message' => 'Reset link sent to email', ]); }
```

### *Task 3.3: Testing with Postman (2 hours)*

**Test Cases to Create in Postman:**

✓ Register new user successfully

✓ Register with duplicate email (should fail)

✓ Login with correct credentials

✓ Login with wrong password (should fail)

✓ Get user profile with token

✓ Get user profile without token (should fail)

✓ Refresh token successfully

✓ Logout successfully

✓ Logout and try to access protected route (should fail)

**Postman Testing Tips:**

→ Create environment with base URL and token variables

→ Use pre-request scripts to automatically save tokens

→ Create folders for each endpoint group

→ Save token from login response for protected routes

→ Test both success and error scenarios

### *Day 3 Deliverables:*

✓ Role-based access control working
✓ Password reset functionality implemented
✓ Complete Postman collection with all tests
✓ All endpoints tested and documented
✓ README with API usage examples

## ■ Quick Reference & Resources

### *Common Commands:*

→ `php artisan make:controller AuthController --api`
→ `php artisan make:resource UserResource`
→ `php artisan migrate`
→ `php artisan tinker` (for testing in terminal)
→ `php artisan serve`

### *Helpful Resources:*

• JWT Explanation
https://jwt.io/introduction

• Laravel JWT Docs
https://github.com/tymondesigns/jwt-auth

• Laravel Docs
https://laravel.com/docs

• Postman Learning
https://learning.postman.com

### *Common Issues & Solutions:*

**Q: Token not working?**
*A: Make sure middleware 'auth:api' is applied to route*

**Q: User not found?**
*A: Check if user exists in database using tinker*

**Q: Email validation fails?**
*A: Make sure validation rule matches request data*

**Q: CORS errors?**
*A: Add 'Accept: application/json' header in Postman*

### *Final Checklist:*

■ All 5+ API endpoints created and working
■ Database migrations created and run
■ User can register, login, and logout
■ Token refresh working
■ Protected routes require token
■ Error messages are clear
■ Postman collection created with examples
■ README with API documentation

Document Created: June 08, 2026